

# ОПЕРАЦИОННЫЕ СИСТЕМЫ

## ЛАБОРАТОРНЫЙ ПРАКТИКУМ

### ЛАБОРАТОРНАЯ РАБОТА № 2.

## Архитектура ОС Windows

#### Цель работы:

изучение архитектуры операционной системы Windows

#### Методические указания к выполнению лабораторной работы

Приложение (application) Windows – это совокупность исполняемых программ и вспомогательных файлов. Например, Microsoft Word представляет собой одно из популярных приложений Windows. Процессом называется исполняемый экземпляр приложения. В большинстве случаев пользователь может запускать несколько экземпляров (копий) одного и того же приложения одновременно. Каждый исполняемый экземпляр – это отдельный процесс со своей собственной областью памяти. Процессом (process) называется исполняемый экземпляр (running in-stance) приложения и комплект ресурсов, отводящийся данному исполняемому приложению. Поток (thread) – это внутренняя составляющая процесса, которой операционная система выделяет процессорное время для выполнения кода. Именно потоки исполняют программный код, а не процессы. Каждый процесс должен иметь как минимум один поток. Конечно, основное назначение потоков – дать процессу возможность поддерживать несколько ветвей управления, то есть выполнять больше действий одновременно. В многопроцессорной конфигурации (компьютер с несколькими процессорами) Windows NT (но не Windows 9x) может распределять потоки по процессорам, реально обеспечивая параллельную обработку. В однопроцессорной конфигурации процессор должен выделять кванты времени (time slices) каждому исполняемому в данный момент потоку. Архитектура ОС Windows базируется на использовании множества различных объектов. Объект ядра (kernel object) – это структура данных, доступ к членам которой, имеет только ядро ОС Windows. Далее приведены примеры объектов ядра: 1. Process представляет процесс; 2. Thread определяет поток; 3. File представляет открытый файл; 4. File-mapping представляет отображаемый в память файл (memorymapped file), то есть файл, содержимое которого отображено непосредственно на виртуальное адресное пространство и используется как физическая память; 5. Pipe используется для обмена данными между процессами; 6. Event является объектом синхронизации потоков, сигнализирующим о завершении операции; 7. Mutex представляет собой объект синхронизации потоков, который может использоваться несколькими процессами; 8. Semaphore используется для того, чтобы учитывать ресурсы и сигнализировать потоку о доступности ресурса на данный момент. Кроме объектов ядра, существуют также пользовательские объекты и объекты GDI, такие как меню, окна, шрифты, кисти и курсоры мыши. Одной из характеристик любого объекта является дескриптор, который используется для идентификации этого объекта. Хотя к объектам ядра нельзя получить непосредственный доступ из пользовательского режима, в Windows API есть функции, которые можно вызывать из данного режима для управления этими объектами. Это своего рода инкапсуляция (encapsulation), защищающая объекты от непредусмотренных или неразрешенных

действий. Когда создается объект ядра посредством вызова соответствующей API функции (CreateProcess, CreateThread, CreateFile и CreateFileMapping), функция возвращает дескриптор вновь созданного объекта. Такой дескриптор может быть передан другой API-функции для того, чтобы она могла управлять данным объектом. Объект файлового отображения можно использовать для межпроцессного взаимодействия наряду с методами, которые будут описаны в других лабораторных работах ниже. Например, для создания объекта файл на базе существующего файла используется API функция: `HANDLE CreateFile( LPCTSTR lpFileName, DWORD dwDesiredAccess, DWORD dwShareMode, LPSECURITY_ATTRIBUTES lpSecurityAttributes, DWORD dwCreationDistribution, DWORD dwFlagsAndAttributes, HANDLE hTemplateFile );` Рассмотрим параметры этой API- функции:

- `lpFileName` – имя открываемого файла;
- `dwDesiredAccess` – `GENERIC_READ`, чтобы разрешить чтение с устройства. `GENERIC_WRITE`, чтобы разрешить запись на устройство (константы можно объединить). Ноль, чтобы разрешить только получение информации об устройстве;
- `dwShareMode` – 0 для запрета общего доступа, `FILE_SHARE_READ` и/или `FILE_SHARE_WRITE` для разрешения общего доступа к файлу;
- `lpSecurityAttributes` – указатель на структуру, определяющую атрибуты безопасности файла (если они поддерживаются операционной системой);
- `dwCreationDistribution` – одна из следующих констант: `CREATE_NEW`: создать файл. Если файл существует, происходит ошибка; `CREATE_ALWAYS`: создать файл. Предыдущий файл перезаписывается; `OPEN_EXISTING`: открываемый файл должен существовать (обязательно используется для устройств) `OPEN_ALWAYS`: создать файл, если он не существует `TRUNCATE_EXISTING`: существующий файл усекается до нулевой длины;
- `dwFlagsAndAttributes` `Long` – комбинация следующих констант: `FILE_ATTRIBUTE_ARCHIVE`: установить архивный атрибут; `FILE_ATTRIBUTE_COMPRESSED`: помечает файл как подлежащий сжатию или задает сжатие для файлов каталога по умолчанию; `FILE_ATTRIBUTE_NORMAL`: другие атрибуты файла не заданы; `FILE_ATTRIBUTE_HIDDEN`: файл или каталог является скрытым; `FILE_ATTRIBUTE_READONLY`: файл доступен только для чтения; `FILE_ATTRIBUTE_SYSTEM`: файл является системным; `FILE_FLAG_WRITE_THROUGH`: операционная система не откладывает операции записи в файл; `FILE_FLAG_OVERLAPPED`: разрешить перекрывающиеся операции с файлом; `FILE_FLAG_NO_BUFFERING`: запретить промежуточную буферизацию файла. Адреса буферов должны выравниваться по границам секторов для текущего тома; `FILE_FLAG_RANDOM_ACCESS`: буферизация файла оптимизируется для произвольного доступа; `FILE_FLAG_SEQUENTIAL_SCAN`: буферизация файла оптимизируется для последовательного доступа; `FILE_FLAG_DELETE_ON_CLOSE`: при закрытии последнего открытого дескриптора файл удаляется. Идеально подходит для временных файлов;
- `hTemplateFile` – если параметр не равен нулю, он содержит дескриптор файла, с которого будут скопированы расширенные атрибуты нового файла. Возвращаемое значение – дескриптор файла в случае успеха.

`INVALID_HANDLE_VALUE` при ошибке. Устанавливает информацию `GetLastError`. Даже если функция завершилась успешно, но файл существовал и был задан флаг `CREATE_ALWAYS` или `OPEN_ALWAYS`, `GetLastError` возвращает `ERROR_ALREADY_EXISTS`.

# Ход выполнения лабораторной работы

## КОД ПРОГРАММЫ:

```
#include <windows.h>
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>

void MirrorFileContent(LPCWSTR filePath) {
    // Step 2: Create file handle
    HANDLE hFile = CreateFile(
        filePath,           // file name
        GENERIC_READ | GENERIC_WRITE, // access mode
        0,                  // share mode
        NULL,               // security attributes
        OPEN_EXISTING,      // creation disposition
        FILE_ATTRIBUTE_NORMAL, // file attributes
        NULL                // template file
    );

    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Error opening file. Error code: " << GetLastError() << std::endl;
        return;
    }

    std::cout << "File handle: " << hFile << std::endl;

    // Step 3: Get file size
    DWORD fileSize = GetFileSize(hFile, NULL);
    if (fileSize == INVALID_FILE_SIZE) {
        std::cerr << "Error getting file size. Error code: " << GetLastError() << std::endl;
        CloseHandle(hFile);
        return;
    }

    std::cout << "File size: " << fileSize << " bytes" << std::endl;

    // Step 4: Create file mapping
    HANDLE hMapping = CreateFileMapping(
        hFile,              // file handle
        NULL,              // security attributes
        PAGE_READWRITE,    // access rights
        0,                 // maximum size high-order dword
        fileSize,          // maximum size low-order dword
        NULL               // name of file mapping object
    );

    if (hMapping == NULL) {
        std::cerr << "Error creating file mapping. Error code: " << GetLastError() << std::endl;
        CloseHandle(hFile);
        return;
    }

    std::cout << "File mapping handle: " << hMapping << std::endl;

    // Step 5: Map file to memory
    LPVOID fileView = MapViewOfFile(
        hMapping,           // file mapping handle
        FILE_MAP_WRITE,    // access mode
        0,                 // file offset high
        0,                 // file offset low
        fileSize            // number of bytes to map
    );

    if (fileView == NULL) {
        std::cerr << "Error mapping file to memory. Error code: " << GetLastError() << std::endl;
        CloseHandle(hMapping);
        CloseHandle(hFile);
        return;
    }

    std::cout << "File mapped address: " << fileView << std::endl;

    // Step 6: Copy data to temporary array
    char* data = new char[fileSize];
    CopyMemory(data, fileView, fileSize);

    // Step 7: Mirror file content
    std::reverse(data, data + fileSize);

    // Step 8: Copy result back to mapped memory
    CopyMemory(fileView, data, fileSize);

    // Step 9: Free memory
    UnmapViewOfFile(fileView);
    CloseHandle(hMapping);
    CloseHandle(hFile);

    delete[] data;
}
```

## РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ:

```
Checking for existence: C:\Documents and Settings\Администратор\Мои документы\Untitled2.exe
Executing: Z:\CodeBlocks\cb_console_runner.exe "C:\Documents and Settings\Администратор\Мои документы\Untitled2.exe" (in C:\Documents and Settings\Администратор\Мои документы)

C:\ "C:\Documents and Settings\Администратор\Мои документы\Untitled2.exe"
Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```

